Domain-Specific Search: BM25 vs. Neural Retrieval on Technical Documents

Intermediate Status Update | Information Retrieval | Group 10 | 25 October 2026

Surajsinh Patil 2025H1030058P Tejas Mahajan 2025H1030057P
Swayam Kumud 2025H1030063P Saksham Singhal 2025H1030208P

Repository: https://github.com/TODO/bm25-vs-neural

1 Implementation status and architecture

We compare our own BM25 implementation with two dense retrievers used without fine-tuning (E5-base-v2 and SPECTER2) and their Reciprocal Rank Fusion hybrids on a frozen corpus of ?? arXiv cs.CR abstracts, using 50 queries in five categories (exact identifier, homonym, negation, time constraint, paraphrase). The whole pipeline is implemented, unit-tested and runs end to end; Table 1 lists the components.

Table 1: Components built so far versus pending. Every built component has unit tests (23 tests: BM25 against the formula, metrics against `pytrec_eval`, kappa against scikit-learn, fusion, pooling).

Component	Status	Notes
Corpus builder (`corpus.py`)	done, frozen	?? docs, SHA-256 `??`; arXiv id list committed
Identifier-aware tokeniser + BM25	done, validated	SciPy sparse matrix, vectorised scoring; matches Lucene (Sec. 3)
Dense encoders + exact search	done	E5 (mean pooling, prefixes), SPECTER2 (proximity / ad-hoc query adapters)
RRF hybrids (k=60)	done	BM25+E5, BM25+SPECTER2 over top-100
Metrics, significance, CIs	done, validated	nDCG, P, MRR, MAP, R, Hole@10; permutation test + Holm; bootstrap CIs
Query set (50 + 5 practice)	done	TREC format; ?? anchors confirmed in corpus, ?? being confirmed
Pooling + blind judging tool	done	?? pairs (≈??/query), 2 judges/query, offline HTML page
Relevance judging	**in progress**	phase 1 (any run's top-5, ?? pairs) first [update]
k1/b sweep, CV, token ablation	code done	runs as soon as the qrels are frozen
Stretch S1–S4	code done	cross-encoder, date filter, LLM judge, CC fusion (Sec. 4)

Changes from the proposal. (1) The Pyserini check builds a Lucene index from exactly the same title+abstract text, so the two BM25s differ only in the implementation. (2) E5 is run through `transformers` with our own mean pooling, because `sentence-transformers` and the `adapters` library needed by SPECTER2 require incompatible library versions; a test checks the pooling against a manual computation. (3) Besides validating BM25 on SciFact we run *all five* systems there, which gives judged results before our own judgments are finished. (4) A judgment-free analysis of top-10 overlap between systems was added. (5) For S2 we first use a rule-based date extractor (no API cost); the LLM extractor is optional. Figure 1 shows the updated architecture.

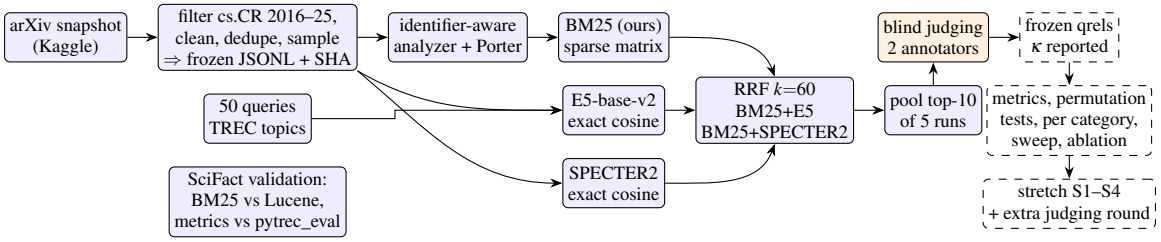


Figure 1: Updated architecture. Shaded = implemented and run; orange = in progress; dashed = code written and tested, waiting for the frozen qrels. Each stage is frozen before the next starts.

2 IR pipeline details

Corpus and preprocessing. From the Kaggle arXiv snapshot we keep papers with cs.CR as primary or cross-listed category and a first version in 2016–2025 (?? cs.CR records in the snapshot, ?? after filtering; ?? withdrawn and ?? duplicate titles removed), strip LaTeX, apply NFKC normalisation and sample ?? documents with a fixed seed (?? have cs.CR as primary category). A document is its title plus abstract.

BM25 (lexical). The identifier-aware analyzer lower-cases, joins protocol versions ("TLS v1.3" → `tls1.3`), keeps identifiers such as `cve-2021-44228`, `sha-256` or `x86-64` as single unstemmed tokens and also indexes their parts, which do not count towards document length (like Lucene's overlapping tokens). Other words pass through Lucene's 33 English stopwords and the Porter stemmer. Note that "no" and "not" are stopwords, so BM25 cannot see negation at all. Term frequencies are stored once in a SciPy CSC matrix (?? terms); for given (k_1, b) the whole BM25-weighted matrix is computed in one vectorised step with the Lucene IDF $\ln(1 + \frac{N - n_t + 0.5}{n_t + 0.5})$, and a query is scored with one sparse matrix–vector product. The main run uses k_1=1.2, b=0.75.

Dense retrieval. Both models are BERT-base (768 dimensions), used as published. E5 uses the `query:`/`passage:` prefixes and mean pooling; SPECTER2 encodes "title [SEP] abstract" with its proximity adapter and queries with its ad-hoc query

adapter ([CLS] vector). Documents are sorted by length for batching (fp16 on a ??); ??% (E5) and ??% (SPECTER2) of the documents exceed 512 tokens and are truncated. Embeddings are cached and searched exactly (brute-force cosine), so no ANN approximation error enters the comparison. **Hybrids** use RRF (k=60, fixed beforehand) over the top-100 of each run, so no score normalisation and no tuning on our queries is needed.

Judging and evaluation. The top-10 of all five runs is pooled. Each query goes to two of the four annotators (25 queries, $\approx$750 judgments each, every annotator pair shares queries so κ covers all pairs). The judging page shows documents in random order with no system or rank, shows dates only for time queries, saves progress in the browser and exports JSON. Disagreements are listed for an adjudication meeting and the qrels are then frozen with a checksum. Metrics follow `trec_eval` conventions (ties broken by docid, unjudged = non-relevant, binary metrics at grade ≥ 2). Significance: paired randomisation test (100k permutations, exact enumeration for $n \leq 16$) with Holm–Bonferroni; per-category means get 95% bootstrap CIs.

3 Preliminary evaluation results

Validation (success criteria i and ii). On BEIR SciFact with identical settings (k_1=0.9, b=0.4, Lucene-like analyzer) our BM25 differs from Pyserini by ΔnDCG@10 = ?? (criterion i ??, threshold 0.01) and shares ??% of its top-10 documents. Our metric code reproduces `pytrec_eval` on every run and query (criterion ii ??). The same check on an offline 10,788-document collection gave ΔnDCG@10 = -0.0026, 96.8% top-10 overlap, and a score ratio of 1.900 to Lucene, i.e. exactly the constant (k_1+1) that Lucene omits; the small remaining differences come from Lucene's lossy encoding of document length.

Table 2: BEIR SciFact results.
[generated by `scripts/validate_scifact.py -dense` (Kaggle notebook, section 2)]

First comparison (SciFact). Table 2 gives the first judged comparison of all five systems against the BM25 baseline (nDCG@10: BM25 ??, E5 ??, SPECTER2 ??, BM25+E5 ??, BM25+SPECTER2 ??). SciFact is a scientific but not a security collection, so it shows that the components behave as published; our research questions are answered on the cs.CR collection. [Add one or two sentences on what the SciFact numbers show once they are in.]

cs.CR collection without judgments. Table 3 shows how different the systems' top-10 lists are: BM25 and E5 share ?? of their top-10 on average (?? on identifier queries, ?? on paraphrases), BM25 and SPECTER2 ??, the two dense models ??. Low overlap means the pool is not dominated by one family and that fusion has complementary evidence to combine. Table 4 reports time and memory.

Table 3: Top-10 overlap.
[generated by `scripts/run_overlap.py`]

Table 4: Efficiency.
[generated by `scripts/make_report_macros.py`]

4 Path to final submission

Table 5: Plan for the remaining work (final submission mid-November).

Dates	Work
26–31 Oct	Finish judging (phase 2: ranks 6–10); compute κ (target ≥ 0.6, otherwise revise guidelines and redo affected queries); adjudication meeting; freeze qrels.
1–4 Nov	Main results (RQ1, RQ3, RQ4) with significance tests; per-category analysis with CIs (RQ2); k1/b sweep with 5-fold CV and tokenisation ablation (RQ5); check our pre-registered expectations.
5–9 Nov	Stretch goals S1 (cross-encoder re-ranking of the best run's top-100) and S2 (date filter on time queries); judge their new documents in an extra blind round.
9–12 Nov	Error analysis and case studies per category; S3/S4 if time allows.
12–15 Nov	Final report, figures, release code, queries, qrels and runs; demo preparation.

Stretch objectives. All four are implemented and tested on a synthetic end-to-end run; none has been run on real data yet. **S1** re-ranks the top-100 with `ms-marco-MiniLM-L-6-v2`; we expect gains mainly on negation queries. **S2** parses date ranges ("after its 2018 standardisation" $\rightarrow \geq$2019) and filters on the first-version date; the rules already parse all 10 time queries. **S3** grades the same pool with an LLM through any OpenAI-compatible endpoint and reports κ and Kendall's τ of the system ranking. **S4** tunes a convex combination of min-max normalised scores with stratified 5-fold CV. Order of priority is unchanged: S1, S2, then S3/S4. **Risks.** Judging time (mitigated by phase-1 ordering); queries without a confirmed relevant document (replaced via random abstracts, never via our systems); possible training-data overlap of the dense models with arXiv abstracts (reported as a limitation).

5 Contribution matrix

Member	Components owned and completed to date
Surajsinh Patil	[e.g. BM25 implementation, identifier-aware tokeniser, validation against Pyserini]
Tejas Mahajan	[e.g. corpus builder and query set]
Swayam Kumud	[e.g. dense encoders, GPU encoding, RRF]
Saksham Singhal	[e.g. evaluation, significance tests, judging tool]
All	Query writing and review, relevance judging (two annotators per query)